Price-Search Prototype: Refined Plan, Module Split and Co-Development Principles

---

Your revision fixes the biggest earlier problems. A few parts would still break in practice, so I'll cover those first, then a refined plan, the module split, and the standard working rules.

What to fix

Ranking per country loses good offers. If each country keeps only its single best product (B.4), your final top 5 can never contain two offers from the same country. If Germany has three of the five cheapest offers, two disappear. Collect all candidates first, then rank them together across countries.

27 countries × 15 pages is too much checking. That's about 400 page visits per search. It means probably more than ten minutes of waiting, hundreds of LLM calls, and repeated visits to the same big shops that appear in several countries. Instead:

- Search broadly: make one SERP call per country. A single call can return up to 100 results.
- Sort by price: Google's top 15 are ordered by relevance and ads, not price, but providers let you sort results by price, low to high.
- Verify narrowly: filter to the exact product, merge duplicates, and open pages only for the ∼15 cheapest candidates overall.

Open the shop's own page, not Google's. "Click into" should mean opening the shop's product page directly. Some results link to a Google product page instead of the shop. In those cases, use the provider's offers data to get the shop's address: Google now serves seller offers through a separate "immersive" product view. This keeps the extension off Google's pages entirely.

Hidden discounts: record them, don't apply them.

- Testing codes means adding items to carts and typing codes at checkout. That breaks easily, and shops can see it as abuse.
- Sign-up discounts would require the agent to create accounts with the user's email without asking. They also only apply to genuinely new customers, which the agent can't know.

For the prototype, detect promotions shown on the page and display them as "possible extra savings" with their conditions. Rank by the guaranteed total, so the ranking stays honest.

Use the LLM as an extractor, not a free-roaming agent. Opening a page doesn't need an LLM. The LLM is most useful for two jobs: checking "is this the same product?", and pulling size, shipping and promotions out of messy, multilingual page text. Give it a fixed output format, and check that every price it returns actually appears on the page. Shop pages are untrusted text: a page could hide instructions aimed at your agent (this is called prompt injection), so treat page text as data only.

Add the missing pieces: a delivery country input (shipping cost depends on it), a confirmation step after identification, currency conversion, a "ships to Italy?" check, and a trust check against scam

shops.

Refined plan

```
Goal: find the current best total price for one product, delivered to
the user's country. No long-term monitoring.

Inputs (side panel)
- At least one of: product name, product code, photo
- Size (required) + delivery country (default from settings)

A. Identify
  A.1 [LLM] Build a product card: brand, model, colour, style code,
      barcode if visible, size system, search queries per language
  A.2 User confirms or corrects the card before searching

B. Search (server, one SERP call per country, in parallel)
  B.1 Fetch price-sorted results per country
  B.2 Normalise: shop, title, price, currency, shop URL, country, sponsored flag
  B.3 [LLM-assisted] Keep exact matches only; convert to EUR; merge duplicates
  B.4 Construct a table to store the processed product information
  B.5 Select the top 15 cheapest candidates in the table

C. Verify (extension, 2-3 pages at a time)
  C.1 Open the shop's product page (never Google pages)
  C.2 [LLM] Confirm it is the same product
  C.3 Size in stock? (structured page data first, [LLM] fallback)
  C.4 Shipping to delivery country: cost, free-shipping threshold,
      or "doesn't ship" ([LLM] reads shipping page, cached per shop)
  C.5 [LLM] Record visible promotions and their conditions
      (don't apply them)
  C.6 Trust check; CAPTCHA or login wall -> "couldn't verify"

D. Rank and show
  D.1 Guaranteed total = item price + shipping (thresholds applied)
  D.2 Rank by guaranteed total; show possible extra savings separately
  D.3 If the filtered products after step C is more than 5, show the Top 5
  + "couldn't verify" products, each with link, country,
  size OK, shipping, promotions, time checked. If the number is less
  than 5 remove the top 15 items in the table generated in step B and jump to B.5
```

Splitting the work

Your plan describes only the extension, but you still need a small server, because API keys must never ship inside an extension. That server is also the natural line between the two of you. One person owns everything in the browser; the other owns everything behind the server's API. Keep both sides in TypeScript in one repository (a "monorepo") with three folders: `extension`, `server` and `shared`. That way you both use the same type definitions.

Module	Owner	Job	Input → output
Shared contracts	Both	TypeScript types, runtime validation (e.g. zod), test fixtures	—
Side panel UI	Dev 1	Input form, card confirmation, progress, results	User input → query; results → screen
Orchestrator	Dev 1	Runs A–D, concurrency, timeouts, cancel	Query → ranked results
Page verifier	Dev 1	Opens shop pages in a background window, collects page snapshots, detects CAPTCHAs and login walls	Candidate → page snapshot
Identify API	Dev 2	Vision LLM + text → product card	Query → product card
Search API	Dev 2	SERP calls, normalising, matching, EUR conversion	Product card → candidates
Extract API	Dev 2	LLM extraction + validation	Page snapshot → verified facts
Pricing & ranking	Dev 2 (in `shared`)	Totals, thresholds, dedupe, top 5	Verified offers → top 5

Give Dev 1 to whoever is stronger with browser and front-end code. Give Dev 2 to whoever is stronger with APIs and prompts. Pricing and ranking should be pure functions (no network, no browser), so it is the easiest module to test thoroughly.

The data objects in the last column (query, product card, candidate, page snapshot, verified facts) are your contracts. Once you've agreed on them, each person can build and test alone.

Save real examples early: a few SERP responses and snapshots of 10–20 real shop pages. Dev 1 runs the extension against a fake ("mock") server that returns those saved examples. Dev 2 tests extraction on the saved pages without a browser. Neither of you waits for the other, and you avoid paying for API calls during most of development.

Standard co-development principles

1. **Contract first.** Write the shared types and the server's API description together, before coding. After that, change them only through a pull request that both of you approve. Add new optional fields rather than renaming existing ones, so the other person's code doesn't break.
2. **One repository, protected main branch.** Use Git with GitHub. Nobody pushes directly to `main`. Every change goes on a short-lived branch and is merged through a pull request the other person reviews. This workflow is called "GitHub flow" if you want to read more.
3. **Small, frequent merges.** Aim for pull requests that take about 15 minutes to review, and merge at least every few days. Branches that live for weeks cause painful conflicts.
4. **Integrate from day one.** First build a "walking skeleton": side panel → mock server → fake top 5 on screen. Then replace the fakes one module at a time, and demo progress to each other weekly.
5. **Automate style and checks.** Use the same formatter and linter (Prettier, ESLint) and TypeScript strict mode. Add automated checks (e.g. GitHub Actions) that run type checks, lint and tests on every pull request. Style debates then mostly disappear.

6. **Make work visible.** Track tasks in GitHub Issues or on a project board. Every task has one owner and a clear definition of done, for example "returns valid JSON for all 10 saved pages, with tests".

7. **Keep secrets out of Git.**

   - Keep API keys in an ignored `.env` file, and commit a `.env.example` with empty values.
   - Set spending limits on your API accounts.
   - If a key is ever committed, revoke it immediately. Deleting the commit isn't enough.

8. **Write decisions down.** Keep a README that gets a new developer running in 10 minutes, and a short decision log ("we chose this SERP provider because…"). This stops you from re-arguing settled questions.

9. **Clear ownership, open code.** Each owner decides inside their module; contracts and architecture are decided together. Either of you can suggest changes anywhere, but through pull requests, not silent edits. If a disagreement drags on, try the simpler option and measure.

10. **Agree on the non-code things early.** Decide who owns the code and the idea, how you'd split a future company, and what happens if one of you stops. If you work for the same employer, check your employment contracts: some claim side projects built during work hours or on company equipment. I'm not a lawyer, but a one-page written agreement now is much easier than that conversation later.

A realistic first week: spend two days together setting up the repository, tooling and contracts, and collecting saved examples. Then work separately against mocks, and have the walking skeleton running by the end of the week.

比价原型：优化后的方案、模块划分与协作开发原则（中文版）

你修改后的方案已经解决了之前最大的问题。但有几个部分在实际运行中仍会出问题，所以我先说明这些问题，然后给出优化后的方案、模块划分，以及标准的协作规则。

需要修改的地方

按国家分别排名会丢掉好的报价。如果每个国家只保留一个最优商品（B.4），最终的前 5 名里就不可能出现同一个国家的两个报价。假如德国占了最便宜的五个报价中的三个，其中两个就会被丢掉。应先收集所有候选商品，再跨国家统一排名。

27 个国家 × 15 个页面，检查量太大。每次搜索大约要访问 400 个页面。这意味着可能要等十分钟以上，调用数百次 LLM，并且会反复访问同一批出现在多个国家的大型商店。建议改为：

- 广泛搜索：每个国家调用一次 SERP API。一次调用最多可返回 100 条结果。
- 按价格排序：Google 的前 15 名是按相关性和广告排序的，而不是按价格；但 SERP 服务商允许你按价格从低到高排序。
- 精准验证：先筛选出完全相同的商品、合并重复项，然后只打开全局最便宜的约 15 个候选商品的页面。

打开商店自己的页面，而不是 Google 的页面。“点进去”应当指直接打开商店的商品页面。有些结果链接到的是 Google 的商品页面，而不是商店。这时，应使用服务商提供的报价数据来获取商店的网址：Google 现在通过一个单独的“immersive”（沉浸式）商品视图提供卖家报价。这样扩展程序就完全不需要访问 Google 的页面。

隐藏折扣：记录下来，但不要自动使用。

- 测试折扣码意味着要把商品加入购物车，并在结账时输入折扣码。这种做法很容易出错，商店也可能将其视为滥用行为。
- 注册折扣需要智能体在未经询问的情况下用用户的邮箱创建账户。而且这类折扣只适用于真正的新顾客，而智能体无法知道用户是不是新顾客。

在原型中，应识别页面上显示的促销信息，并将其作为“可能的额外优惠”连同使用条件一起展示。排名则按“确定的总价”计算，这样排名才是诚实可信的。

把 LLM 当作信息提取器，而不是自由行动的智能体。打开页面不需要 LLM。LLM 最适合做两件事：判断“这是否是同一件商品?”，以及从杂乱的多语言页面文字中提取尺码、运费和促销信息。要给它固定的输出格式，并核对它返回的每个价格是否真的出现在页面上。商店页面是不可信的文字：页面中可能藏有针对你的智能体的指令（这被称为“提示词注入”），所以只能把页面文字当作数据处理。

补上缺失的部分：收货国家输入项（运费取决于它）、识别之后的确认步骤、货币换算、“是否寄往意大利?”的检查，以及针对诈骗网店的可信度检查。

优化后的方案

```
目标：找出某一商品寄送到用户所在国家的当前最低总价。
不做长期价格监控。

输入（侧边栏）
- 至少提供以下一项：商品名称、商品编码、照片
- 尺码（必填）+ 收货国家（默认取自设置）

A. 识别
  A.1 [LLM] 生成商品卡片：品牌、型号、颜色、款号、
      条形码（如可见）、尺码体系、各语言的搜索关键词
  A.2 搜索开始前，由用户确认或修改商品卡片

B. 搜索（服务器端，每个国家调用一次 SERP API，并行执行）
  B.1 获取每个国家按价格排序的结果
  B.2 统一格式：商店、标题、价格、币种、商店网址、国家、
      是否为广告
  B.3 [LLM 辅助] 只保留完全匹配的商品；换算成欧元；
      合并重复项
  B.4 创建列表按价格升序储存所有候选商品信息。
  B.5 选择前十五名最便宜的商品
C. 验证（扩展程序端，每次 2-3 个页面）
  C.1 打开商店的商品页面（绝不打开 Google 页面）
  C.2 [LLM] 确认是同一件商品
  C.3 该尺码是否有货？(优先读取页面结构化数据，[LLM] 作为后备)
  C.4 寄往收货国家的运费：费用、包邮门槛，或“不寄送”
      （[LLM] 读取运费说明页面，按商店缓存）
  C.5 [LLM] 记录页面上可见的促销及其条件（不自动使用）
  C.6 可信度检查；遇到验证码或登录墙 -> 标记为“无法验证”

D. 排名与展示
  D.1 确定总价 = 商品价格 + 运费（已考虑包邮门槛）
  D.2 按确定总价排名（若有上次循环中保存的商品，需包含）；可能的额外优惠单独展示
  D.3 若经步骤 C 过滤后的商品数量大于 5，则输出商品价格前 5 名 + “无法验证”列表，
  每项包含链接、国家、尺码有货、运费、促销、检查时间。
  若经步骤 C 过滤后的商品数量小于 5，则移除步骤 B 中创建的列表中前十五名的商品，
  并跳转至步骤 B.5
```

分工方式

你的方案只描述了扩展程序，但你仍然需要一个小型服务器，因为 API 密钥绝不能放在扩展程序里发布。这个服务器也自然地成为你们两人之间的分界线：一个人负责浏览器里的所有部分，另一个人负责服务器 API 背后的所有部分。两边都使用 TypeScript，放在同一个代码仓库（“monorepo”，单一

代码仓库）中，分成三个文件夹：`extension`、`server` 和 `shared`。这样你们俩可以使用同一套类型定义。

模块	负责人	职责	输入 → 输出
共享约定	两人共同	TypeScript 类型、运行时校验（如 zod）、测试样例数据	—
侧边栏界面	开发者 1	输入表单、卡片确认、进度、结果	用户输入 → 查询；结果 → 界面
流程编排器	开发者 1	运行 A–D 步骤、并发控制、超时、取消	查询 → 排名结果
页面验证器	开发者 1	在后台窗口打开商店页面、收集页面快照、识别验证码和登录墙	候选商品 → 页面快照
识别 API	开发者 2	视觉 LLM + 文字 → 商品卡片	查询 → 商品卡片
搜索 API	开发者 2	SERP 调用、统一格式、匹配、欧元换算	商品卡片 → 候选商品
提取 API	开发者 2	LLM 信息提取 + 校验	页面快照 → 已验证信息
定价与排名	开发者 2（放在 `shared` 中）	总价计算、门槛、去重、前 5 名	已验证报价 → 前 5 名

让更擅长浏览器和前端代码的人担任开发者 1，让更擅长 API 和提示词的人担任开发者 2。定价与排名应写成纯函数（不涉及网络和浏览器），因此它是最容易进行全面测试的模块。

表格最后一列中的数据对象（查询、商品卡片、候选商品、页面快照、已验证信息）就是你们之间的“约定”。一旦双方达成一致，每个人都可以独立开发和测试。

尽早保存真实样例：几份 SERP 返回结果，以及 10–20 个真实商店页面的快照。开发者 1 让扩展程序连接一个返回这些样例的假服务器（“mock” 服务器）来运行；开发者 2 在不打开浏览器的情况下，用保存的页面测试信息提取。这样谁都不用等对方，而且在大部分开发过程中都不必为 API 调用付费。

标准协作开发原则

1. **约定优先**。在写代码之前，共同写好共享类型和服务器 API 说明。之后只能通过双方都批准的拉取请求（pull request）来修改它们。尽量新增可选字段，而不是重命名已有字段，以免破坏对方的代码。
2. **一个代码仓库，受保护的主分支**。使用 Git 和 GitHub。任何人都不直接推送到 `main` 分支。每项改动都在一个短期分支上完成，并通过由对方审查的拉取请求合并。如果想深入了解，这种工作流程叫做 “GitHub flow”。
3. **小步提交，频繁合并**。每个拉取请求最好在 15 分钟左右就能审查完，并且至少每隔几天合并一次。存在好几周的分支会导致痛苦的代码冲突。
4. **从第一天就开始集成**。先搭建一个 “行走骨架”（walking skeleton）：侧边栏 → mock 服务器 → 在界面上显示假的前 5 名。然后逐个模块替换掉假的部分，并且每周互相演示进展。
5. **把代码风格和检查自动化**。使用相同的格式化工具和代码检查工具（Prettier、ESLint），并开启 TypeScript 严格模式。添加自动化检查（如 GitHub Actions），在每个拉取请求上运行类型检查、

代码检查和测试。这样关于代码风格的争论基本就会消失。

6. **让工作进度可见**。用 GitHub Issues 或项目看板跟踪任务。每项任务都有一个负责人和清晰的“完成标准”，例如“对全部 10 个保存的页面都返回合法的 JSON，并附带测试”。
7. **不要把密钥放进 Git**。
   - 把 API 密钥放在被 Git 忽略的 `.env` 文件中，并提交一个值为空的 `.env.example`。
   - 为你们的 API 账户设置消费上限。
   - 如果密钥曾被提交，立即作废它。仅仅删除那次提交是不够的。
8. **把决定写下来**。维护一份能让新开发者在 10 分钟内跑起项目的 README，以及一份简短的决策记录（“我们选择这个 SERP 服务商是因为……”）。这能避免对已经定下的问题反复争论。
9. **职责清晰，代码开放**。每个负责人在自己的模块内做决定；约定和架构由两人共同决定。任何一方都可以对任何地方提出修改建议，但要通过拉取请求，而不是悄悄修改。如果分歧迟迟无法解决，就先试更简单的方案，再用数据衡量。
10. **尽早商定代码之外的事情**。确定代码和创意归谁所有、将来如果成立公司如何分配，以及如果其中一人退出该怎么办。如果你们受雇于同一家公司，请检查各自的劳动合同：有些合同会主张对在工作时间内或使用公司设备开发的副业项目拥有权利。我不是律师，但现在写一份一页纸的书面协议，要比以后再谈这个问题容易得多。

一个现实的第一周安排：前两天一起搭建代码仓库、工具链和约定，并收集样例数据。之后各自基于 mock 独立开发，在周末前让“行走骨架”跑起来。